

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài đã nghiên cứu, thiết kế và xây dựng hệ điều hành thời gian thực nhúng MyOS trên kiến trúc ARM Cortex-M4F, hướng đến vi điều khiển STM32F407. Quá trình triển khai từ mức kernel và architecture port đã làm rõ các nguyên lý cốt lõi của RTOS, gồm quản lý vòng đời task, lập lịch theo độ ưu tiên, chuyển ngữ cảnh, quản lý thời gian, đồng bộ hóa, giao tiếp liên tác vụ và quản lý bộ nhớ.

Mã nguồn MyOS được tổ chức thành các lớp application, kernel, architecture port, board/platform, HAL và driver. Cách tổ chức này đã tạo được ranh giới tương đối rõ giữa phần phụ thuộc kiến trúc Cortex-M4F và phần logic của kernel, qua đó hỗ trợ việc kiểm thử, bảo trì và mở rộng hệ thống.

Các kết quả chính đã đạt được gồm:

- Quản lý task với các thao tác tạo tĩnh hoặc động, delay, yield, suspend, resume, kill và exit; đồng thời theo dõi trạng thái và thông tin stack của từng task.
- Lập lịch ưu tiên có cường bức, kết hợp round-robin và time slicing cho các task cùng mức ưu tiên.
- Khởi động task đầu tiên thông qua SVC và thực hiện chuyển ngữ cảnh bằng PendSV; phần context switch có cơ chế lưu và khôi phục các thanh ghi dấu phẩy động mở rộng khi task sử dụng FPU.
- Quản lý thời gian bằng SysTick, hỗ trợ delay, timeout và software timer một lần hoặc định kỳ.
- Đồng bộ hóa bằng binary semaphore, counting semaphore, mutex, recursive mutex và cơ chế priority inheritance.
- Giao tiếp liên tác vụ bằng message queue, message buffer và event group; một số thao tác có biến thể sử dụng trong ngữ cảnh ISR.
- Quản lý heap động có tách, gộp block và thống kê dung lượng trống, mức thấp nhất từng ghi nhận, block trống lớn nhất và tỷ lệ phân mảnh.
- Theo dõi stack bằng vùng guard, stack canary và high-water mark.

- Tích hợp cấu hình MPU tĩnh và vùng MPU động theo task, fault handler, trace buffer, runtime statistics, UART shell và tickless idle ở mức kernel.
- Xây dựng ứng dụng minh họa điều khiển nhiệt độ sử dụng cảm biến mô phỏng, message queue, mutex, UART shell và GPIO; đồng thời xây dựng bộ kịch bản kiểm thử chức năng và quy trình chạy trên Renode.

Kết quả kiểm tra trong mã nguồn và các log Renode cho thấy những cơ chế đại diện như round-robin, priority inheritance và event group đã hoạt động theo kịch bản kiểm thử. Tuy nhiên, kết quả hiện tại phù hợp nhất để khẳng định MyOS là một RTOS phục vụ nghiên cứu và thực nghiệm, chưa đủ cơ sở để xem là một hệ điều hành sẵn sàng cho ứng dụng sản phẩm.

Giá trị quan trọng của đề tài là làm rõ mối liên hệ giữa scheduler, trạng thái task, stack, exception, interrupt và cơ chế chuyển ngữ cảnh trên Cortex-M4F. Việc tự xây dựng các thành phần này giúp người thực hiện hiểu sâu hơn cách một RTOS vận hành từ mức thanh ghi phần cứng đến API ứng dụng.

6.2. Hạn chế của hệ thống

Mặc dù đã triển khai được các thành phần chính, MyOS vẫn còn các hạn chế sau:

- Hệ thống mới được kiểm thử chủ yếu bằng các kịch bản chức năng trong môi trường mô phỏng. Chưa có đủ dữ liệu về độ ổn định khi vận hành dài hạn và khi chịu tải trên phần cứng STM32F407 thực.
- Các task hiện khởi động ở Thread mode sử dụng PSP nhưng vẫn chạy đặc quyền. MyOS chưa có mô hình task không đặc quyền và hệ thống system call hoàn chỉnh thông qua SVC.
- MPU đã có cấu hình vùng tĩnh, vùng stack và vùng mở rộng theo task, nhưng chưa hình thành cơ chế cô lập hoàn chỉnh giữa task và kernel; các bài kiểm thử truy cập sai vùng nhớ và MemManage Fault còn thiếu.
- Mã chuyển ngữ cảnh đã xử lý phần ngữ cảnh FPU mở rộng, nhưng chưa có bộ kiểm thử chuyên biệt chứng minh việc bảo toàn dữ liệu dấu phẩy động giữa nhiều task trong mọi trường hợp.

- Tickless idle đã được bật và có cơ chế dừng hoặc lập trình lại SysTick, nhưng chưa được đánh giá đầy đủ về sai số thời gian, tình huống đánh thức biên và mức tiết kiệm năng lượng trên phần cứng thực.
- Ranh giới HAL, driver và application chưa hoàn toàn chặt chẽ. Một số module kernel và task ứng dụng vẫn phụ thuộc trực tiếp vào UART hoặc gọi API ngoại vi cụ thể.
- Cơ chế log và chẩn đoán còn phụ thuộc vào UART polling ở một số vị trí, có thể làm tăng thời gian xử lý và ảnh hưởng đến tính xác định của hệ thống.
- Bộ kiểm thử hiện chưa bao phủ đầy đủ message buffer, MPU, FPU, tickless idle, các tình huống đồng thời phức tạp, tick wrap-around và vận hành thời gian dài.
- Các chỉ tiêu thời gian thực như thời gian chuyển ngữ cảnh, độ trễ ngắt, scheduler latency và jitter của task định kỳ chưa được đo đạc định lượng trên phần cứng thực.
- Hệ thống chưa có file system, giao thức mạng, device model, cơ chế quản lý driver thống nhất và các thành phần đảm bảo độ tin cậy ở mức sản phẩm như watchdog supervisor hoặc crash dump bền vững.
- Ứng dụng minh họa hiện dùng dữ liệu nhiệt độ mô phỏng; do đó chưa chứng minh đầy đủ khả năng tích hợp cảm biến và luồng I/O bất đồng bộ trên phần cứng thực.

Những hạn chế trên không làm thay đổi mục tiêu nghiên cứu của đề tài, nhưng cần được nêu rõ để phạm vi kết luận phù hợp với mức độ triển khai và bằng chứng kiểm thử hiện có.

6.3. Hướng phát triển

Trong thời gian tới, MyOS có thể được tiếp tục hoàn thiện theo các hướng sau:

- Hoàn thiện mô hình task không đặc quyền và xây dựng lớp system call thông qua SVC, bao gồm kiểm tra tham số và xác thực con trỏ từ task.
- Hoàn thiện cơ chế MPU theo task; bổ sung kiểm thử truy cập sai vùng nhớ, stack guard, vùng XN, MemManage Fault và khả năng cô lập lỗi.

- Xây dựng bài kiểm thử chuyên biệt cho việc lưu và khôi phục ngữ cảnh FPU giữa nhiều task sử dụng phép toán dấu phẩy động.
- Chuẩn hóa kiến trúc HAL và driver để application và kernel không phụ thuộc trực tiếp vào UART hoặc thanh ghi ngoại vi.
- Xây dựng giao diện log độc lập với backend, cho phép lựa chọn UART, SWO hoặc bộ đệm RAM; hạn chế log đồng bộ trong critical section và ISR.
- Bổ sung driver UART bất đồng bộ, I2C, SPI, ADC, timer, PWM, DMA, RTC và watchdog; sau đó thay cảm biến mô phỏng bằng cảm biến thực.
- Đo thời gian chuyển ngữ cảnh, độ trễ ngắt, scheduler latency, độ trễ đánh thức task và jitter bằng DWT cycle counter, GPIO và logic analyzer trên STM32F407.
- Hoàn thiện tickless idle, xử lý các trường hợp biên và đo mức tiêu thụ năng lượng thực tế ở các chế độ hoạt động khác nhau.
- Mở rộng kiểm thử tự động cho toàn bộ API, kiểm thử tải, kiểm thử thời gian dài, tick wrap-around, nhiều timeout đồng thời và các lỗi cạnh tranh; tích hợp quy trình build và test vào CI.
- Bổ sung crash dump, watchdog supervisor, assert kiểm tra tính nhất quán nội bộ và cơ chế lưu thông tin fault để tăng khả năng chẩn đoán.
- Nghiên cứu và thử nghiệm các thuật toán lập lịch thời gian thực như Rate Monotonic Scheduling và Earliest Deadline First sau khi đã có bộ benchmark và mô hình deadline phù hợp.
- Chuyển architecture port sang các vi điều khiển Cortex-M khác để đánh giá mức độ độc lập phần cứng và khả năng tái sử dụng của kernel.
- Sau khi kernel và driver ổn định, xem xét tích hợp file system và TCP/IP stack phù hợp với tài nguyên của vi điều khiển.

Nhìn chung, MyOS đã tạo được nền tảng ban đầu cho một hệ điều hành thời gian thực nhúng trên ARM Cortex-M4F. Hướng phát triển phù hợp nhất là ưu tiên tính đúng đắn, khả năng kiểm chứng và an toàn của kernel trước khi mở rộng middleware. Với lộ trình đó, MyOS có thể trở thành một nền tảng nghiên cứu, giảng dạy và thực nghiệm có giá trị, đồng thời tiến gần hơn đến khả năng vận hành tin cậy trên phần cứng thực.